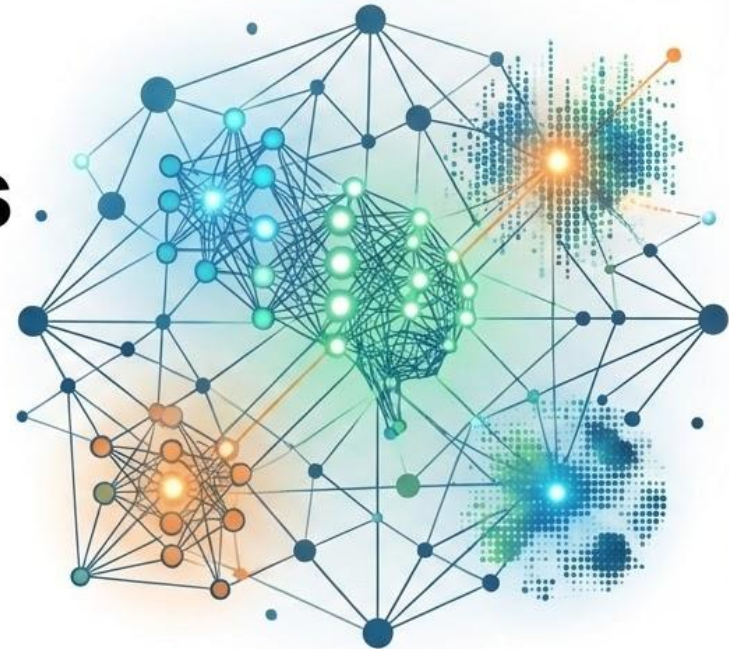


Diplomatura en python para la ciencia de datos y machine learning

Módulo 1 – Desarrollo Avanzado en Python

7- Análisis de Datos con Pandas

Temas: Series y DataFrames, Carga y exportación de datos, EDA (Exploratory Data Analysis), Indexación, selección y filtrado



¿Qué es Pandas?

¿Qué es Pandas?

Definición: Pandas es una biblioteca de software (paquete) que agrega funcionalidades a Python para la manipulación y el análisis de datos.

```
import pandas as pd
import numpy as np
```

Usos Principales: Se utiliza principalmente para la limpieza de datos, análisis exploratorio, y visualización de datos estadísticos.

Conceptualización: La Caja de Herramientas de Datos.

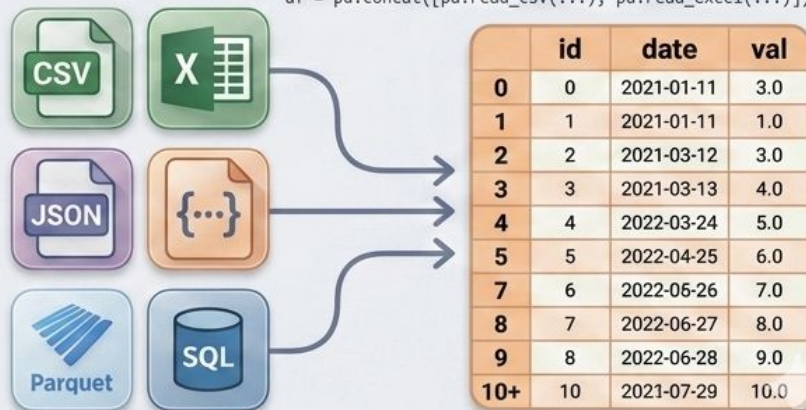


Importancia de Pandas en Data Science

- **Facilita el Manejo de Datos:** Pandas simplifica el manejo de datos, permitiéndonos enfocarnos en el análisis.
- **Adaptado a Diferentes Fuentes de Datos:** CSV, Excel, SQL, entre otros.

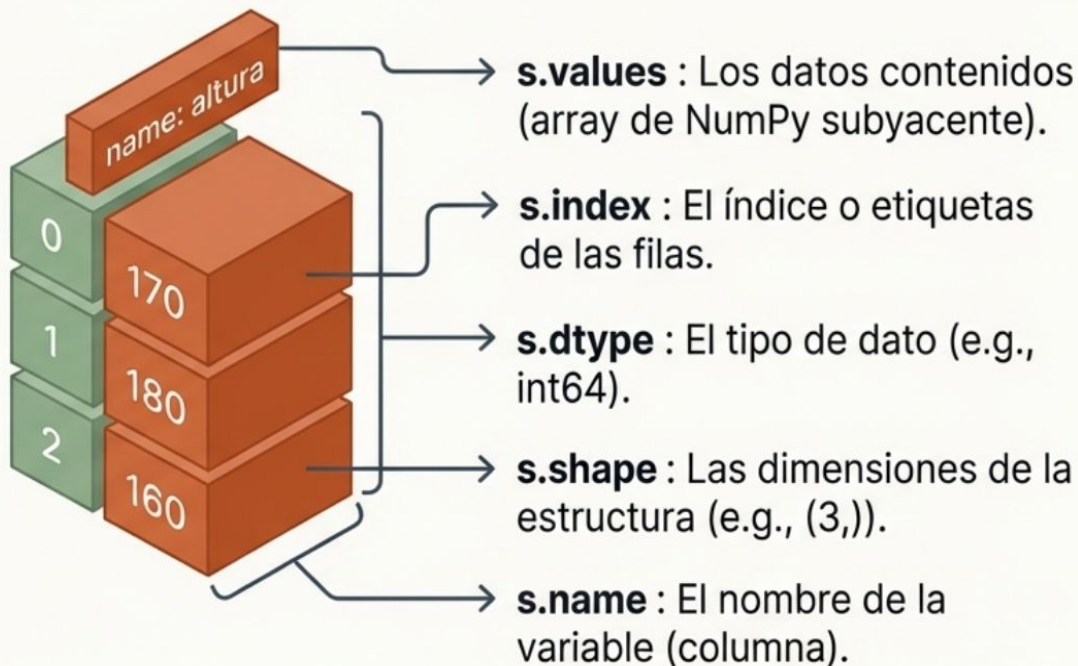
Capacidad de Integración y Procesamiento.

```
df = pd.concat([pd.read_csv(...), pd.read_excel(...)])
```



Anatomía de una Estructura (La Serie)

```
s = pd.Series([170, 180, 160],  
              name='altura')
```



Idea Clave: Una Series es fundamentalmente una columna enriquecida con metadatos.

Ingesta: Dando Vida al DataFrame

Desde un Diccionario (Programático)

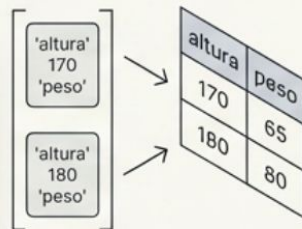
```
df = pd.DataFrame({  
    'altura': [170, 180],  
    'peso': [65, 80]  
})
```



	altura	peso
1	170	65
2	180	80

Desde una Lista de Diccionarios (APIs)

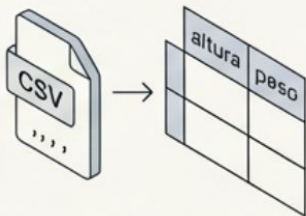
```
df = pd.DataFrame([  
    {'altura': 170, 'peso': 65},  
    {'altura': 180, 'peso': 80}  
])
```



	altura	peso
	170	65
	180	80

Desde Archivos CSV (Estándar)

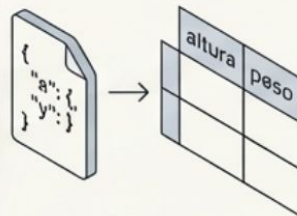
```
df = pd.read_csv('datos.csv')
```



	altura	peso

Desde Archivos JSON (Datos anidados)

```
df = pd.read_json('datos.json')
```



	altura	peso

(También compatible con Excel mediante `pd.read_excel('datos.xlsx', sheet_name='Hoja1')`)

Pandas Data Types

ESTRUCTURA Y CLASIFICACIÓN DE COLUMNAS EN DATAFRAMES



int64

Valores enteros (discretos) sin decimales.

Ej: edad: 30



float64

Números con decimales para valores continuos.

Ej: precio: 19.50



object

Texto o tipos mixtos. Gran flexibilidad.

Ej: ciudad: "Madrid"



bool

Lógica binaria de verdadero o falso.

Ej: activo: True



datetime64

Puntos específicos en el tiempo (fechas/horas).

Ej: 2024-01-01



timedelta64

Diferencias o duraciones de tiempo.

Ej: 2 días 04:00



category

Datos categóricos. Optimiza memoria eficientemente.

Ej: sexo, estado_civil

RESUMEN: ÍNDICES Y MANIPULACIÓN EN PANDAS

ÍNDICES: EL EJE DE LAS FILAS



ÍNDICE:

Únicas (por defecto),
Cadenas (nombres),
Fer (Series de Tiempo).

- **DEFINICIÓN:**
Etiquetas únicas que identifican cada fila.
- **FUNCIONES:**
Facilita acceso rápido, alineación de datos y unión de tablas.
- **TIPOS COMUNES:**
Numéricos (por defecto), Cadenas (nombres), Fechas

index

$\begin{bmatrix} 0, \\ 1, 1 \\ 1, 2 \end{bmatrix} \rightarrow \begin{bmatrix} 'A', \\ 'B', \\ 'C' \end{bmatrix}$

MANIPULACIÓN DE ÍNDICES: OPERACIONES CLAVE



ESTABLECER ÍNDICE (df.set_index)

Convierte una columna de datos en el índice del DataFrame.
Ej: Columna 'ID' -> Nuevo Índice.



RESTABLECER ÍNDICE (df.reset_index)

Vuelve el índice actual a una columna de datos y crea un nuevo índice numérico secuencial.



REINDEXAR Y RENOMBRAR

Ajustar las etiquetas (reindex) o cambiar sus nombres (rename). Útil para orden y claridad.

Cargando datos: read_csv en profundidad

El 90% del tiempo vas a cargar datos desde archivos de texto — conocer sus parámetros ahorra horas de trabajo

● Caso más común: leer un archivo local

```
df = pd.read_csv('ventas.csv')

# Con parámetros útiles
df = pd.read_csv(
    'ventas.csv',
    sep=';',          # separador punto y coma
    encoding='utf-8', # codificación del archivo
    nrows=1000,       # leer solo 1000 filas
    usecols=['fecha', 'monto'], # solo esas columnas
    parse_dates=['fecha'], # parsear como fecha
    na_values=['N/A', '-'] # tratar como nulos
)
```

● Leer directamente desde una URL

```
url = 'https://raw.githubusercontent.com/.../titanic.csv'
df = pd.read_csv(url) # funciona igual que un archivo local
```

OTROS FORMATOS DISPONIBLES

Excel

read_excel()

JSON

read_json()

SQL

read_sql()

Parquet

read_parquet()

Todos comparten la misma lógica: devuelven un DataFrame listo para trabajar. La diferencia está en el formato del archivo de origen.

Exportar también es posible

```
df.to_csv('resultado.csv', index=False)
df.to_excel('resultado.xlsx')
df.to_json('resultado.json')
```

Usá `index=False` en `to_csv` para evitar que guarde el índice numérico como una columna extra.

Error frecuente

Si el archivo tiene tildes o ñ y aparecen caracteres raros, agregá `encoding='latin-1'` o `encoding='utf-8-sig'`. Es el problema de codificación más común con archivos en español.

EDA: Inspección rápida con df.info()

Obtén una visión instantánea de la estructura y salud de tus datos

◆ Muestra un resumen general del DataFrame

```
df.info()
```

Conceptualización:
Escáner de Salud del Dataset

✓ Beneficios clave:

- ✓ Cantidad de filas y columnas (RangeIndex y column counts).
- ✓ Nombres de columnas.
- ✓ Tipos de datos (int, float, object, etc.).
- ✓ Valores no nulos por columna (Non-null counts).
- ✓ Uso de memoria.



✓ Diagnóstico instantáneo de integridad

◆ ¿Para qué sirve?

Detectar valores faltantes
Ver tipos incorrectos
Tener una visión rápida de la estructura

◆ Ejemplo de interpretación

Si una columna tiene menos “non-null” que el total → hay nulos.

salario	(Float64)	(Total
dilatas	1000	1000 > 980
non-null	980	-> 20 Nulos)

valores = ["Estructura", "Salud", "Limpieza"]

EDA (Análisis Exploratorio de Datos) en Pandas

Antes de escribir cualquier transformación, siempre hay que entender qué tenemos

(1) Estructura ✓

```
df.head()    # primeras 5 filas
df.tail(3)   # últimas 3 filas
df.shape     # (filas, columnas)
df.columns   # nombres de columnas
df.dtypes    # tipo por columna
```

Conceptualización: Plano de Arquitectura de Datos

RangeIndex: 200, entry, 0 to 199

nombre	object	200 non-null
edad	int64	200 non-null
salario	Float64	180 non-null
ciudad	object	200 non-null

Conceptualización: Plano de Arquitectura de Datos

- ✓ Si **non-null** es menor que el total de **filas** -> hay valores nulos en esa columna.



(2) Estadísticas ✓

```
df.describe()          # numéricas
df.describe(include='all') # todas
```

	count	edad	salario
count	180	maan	54.8
min	22.000	int6k	98
max	41.200	stda	
25%	41.200	75%	67.3
mean	11.500	titda	13.300



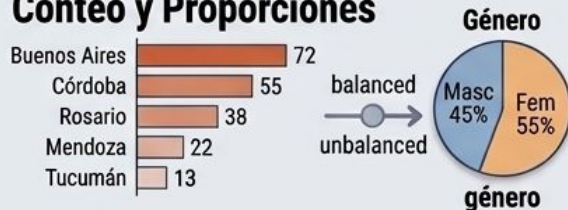
Conceptualización: Resumen Numérico Instantáneo

- ✓ **count** menor al total -> hay nulos
- ✓ **min / max** muy alejados de la media -> posibles outliers
- ✓ **mean** muy diferente al 50% -> distribución sesgada

(3) Frecuencias ✓

```
df['ciudad'].value_counts()
# con porcentajes:
df['ciudad'].value_counts(normalize=True)
# valores únicos:
df['ciudad'].unique()
```

Conceptualización: Conteo y Proporciones



- ✓ Un **desbalance** muy grande entre categorías puede afectar al modelo — hay que detectarlo acá.
 - ✓ Conteo de valores
 - ✓ Proporciones (%)
 - ✓ Detectar clases desbalanceadas
- valores = ["Ciudad", "Género"]

¿Filtrado de datos en Pandas?

df.query() vs df.isin()

◆ df.query()

Permite filtrar filas usando expresiones tipo SQL (más legible).

```
df.query("edad > 30 and ciudad == 'San Luis'")
```

- ✓ Más claro para condiciones complejas
- ✓ Evita escribir muchos corchetes
- ⚠ Usa strings (cuidado con nombres de columnas)

Conceptualización: El Ojo Crítico del SQL.



◆ df.isin()

Filtra por pertenencia a un conjunto de valores.

```
df[df["ciudad"].isin(["San Luis", "Mendoza"])]
```

- ✓ Ideal para múltiples valores posibles
- ✓ Muy útil en filtros categóricos

◆ Diferencia Clave

query(): condiciones lógicas generales |

isin(): pertenencia a listas/conjuntos



El Bisturí: Selección y Filtrado

Acceso Directo

- **Por etiqueta (loc):** Usa el índice explícito. Útil cuando el índice tiene significado (IDs, fechas).

```
s.loc['b'] \  
df.loc[5]
```

- **Por posición (iloc):** Usa posición numérica. Para trabajar como si fuera una matriz (estilo NumPy).

```
s.iloc[1] \  
df.iloc[0:3]
```

Filtrado Condicional (Booleano)

La base del preprocesamiento antes del modelo.

```
df[df['edad'] > 30]
```

Condiciones múltiples (usar & para AND, | para OR):

```
df[(df['edad'] > 30) & (df['ingreso'] > 50000)]
```

Mensajes de error que verás a menudo por uso incorrecto de indexadores.

```
# ERROR 1: Mezcla de .loc con .iloc  
# .loc espera etiquetas, .iloc espera posiciones  
df.loc[0, [0]] # ¿Es posición o etiqueta?  
# Mensaje de Error: "KeyError: 0"
```

```
# ERROR 2: Índice fuera de rango con .iloc  
# .iloc espera posición numérica válida  
s.iloc[1000] # El índice es demasiado grande  
# Mensaje de Error: "IndexError: single positional i
```